Stereo Matching and 3D Reconstruction

NTU Computer Vision Final Project – Team 10

Team 10

National Taiwan University

Spring 2026

Outline

- Basic + bonus summary
- Basic implementation: `computeDisp.py`
- Basic results + runtime analysis
- Bonus implementation: disparity $\rightarrow$ depth $\rightarrow$ TSDF mesh
- Bonus demo: first-person walkthrough

Quick summary

- **Basic:** classical stereo matcher in `computeDisp.py`
- **Bonus:** use predicted disparity for TartanAir 3D reconstruction
- **Output:** disparity maps, mesh, walkthrough video

Basic: actual code path

- Start from grayscale for Census
- Build **left** cost volume
- Get **right** cost by reindexing
- Aggregate left cost only
- Final output: left disparity map

```
computeDisp(Il, Ir, max_disp):
  gray_L = BGR2GRAY(Il)
  gray_R = BGR2GRAY(Ir)

  cL = census_transform(gray_L)
  cR = census_transform(gray_R)

  cost_L = hamming_cost_volume(cL, cR)
  cost_R = reindex_cost_to_right(cost_L)

  cost_L = aggregate(cost_L, Il)

  D_L = argmin(cost_L, axis=d)
  D_R = argmin(cost_R, axis=d)

  labels = refine(D_L, D_R, cost_L, Il)
  return uint8(labels)
```

Step 1: Census transform

- Window: 9×7
- $9 \times 7 - 1 = 62$ comparisons
- Stored in one `uint64`
- Border: replicated padding
- More stable than raw intensity difference

$$\text{bit}_i = [I(\text{neighbor}_i) < I(\text{center})]$$

```
census_transform(gray):
  pad image by edge values
  code = zeros(H, W, uint64)

  for each offset in 9x7 window:
    if offset is center: continue
    bit = (neighbor < center)
    code |= bit << bit_index

  return code
```

Step 1: Hamming cost volume

- Cost = bit differences
- Shape: $(D+1) \times H \times W$
- OOB columns: clamp to border
- Popcount path:
  - `np.bitwise_count`, if available
  - SWAR fallback, otherwise
- Parallel over disparity slices

$$C(d, y, x) = \text{popcount}(c_L(y, x) \oplus c_R(y, x-d))$$

```
hamming_cost_volume(cL, cR):
  for d = 0 ... max_disp parallel:
    xR = clip(x - d, 0, W-1)
    xor = cL[y, x] XOR cR[y, xR]
    cost[d, y, x] = popcount(xor)

  return cost  # (D+1, H, W)
```

Optimization: right cost by reindexing

- Need `D_R` only for LRC mask
- Hamming: symmetric matching cost
- Avoids another Hamming pass
- `cost_R` stays **raw**, not WGIF-filtered

$$C_R(d, y, x_R) = C_L(d, y, \text{clip}(x_R + d))$$

```
reindex_cost_to_right(cost_L):
  for each disparity d:
    for each right column xR:
      xL = clip(xR + d, 0, W-1)
      cost_R[d, y, xR] = cost_L[d, y, xL]

  return cost_R
```

Step 2: WGIF aggregation

- Guide: left image grayscale
- Filter each disparity cost slice
- Adaptive ϵ through gamma
- Edges: preserve discontinuities
- Flat regions: smooth stronger
- Parallel over slices again

$$q(x) = \bar{a}(x)I(x) + \bar{b}(x)$$

```
aggregate(cost, guide):
  I = grayscale(guide) / 255
  mean_I = boxFilter(I)
  var_I  = boxFilter(I*I) - mean_I^2

  gamma = (var_I + eps0) / mean(var_I + eps0)
  denom = var_I + eps / gamma

  for each disparity slice p parallel:
    mean_p  = boxFilter(p)
    cov_Ip  = boxFilter(I*p) - mean_I*mean_p
    a = cov_Ip / denom
    b = mean_p - a*mean_I
    q = boxFilter(a)*I + boxFilter(b)
```

Step 3: WTA selection

- One disparity per pixel
- D_L: final candidate disparity
- D_R: consistency helper
- Skips right-side aggregation

$$D_L(y, x) = \arg\min_d C_L(d, y, x)$$

```
winner_take_all(cost):
  return argmin(cost, axis=disparity)

D_L = WTA(filtered cost_L)
D_R = WTA(raw cost_R)
```

Step 4: Refinement: LRC + hole filling

- Check left/right agreement
- Invalid if:
  - matched x out of bound
  - disparity difference > 1
- Fill holes row by row
- Use nearest valid from left/right
- Final fill = smaller one

$$|D_L(y,x) - D_R(y, x - D_L(y,x))| \leq 1$$

```
valid = LR_consistency(D_L, D_R)

for each row:
  left_fill  = sweep_left_to_right()
  right_fill = sweep_right_to_left()

filled = min(left_fill, right_fill)
```

Step 4: Refinement: sub-pixel + weighted median

- Valid pixels:
  - refine integer d by parabola fitting
  - use cost at $d-1, d, d+1$
- Invalid pixels:
  - use hole-filled disparity
- Then weighted median smoothing
- Guide image: quantized left BGR
- Final output is `uint8`

```
d_sub = subpixel_refine(D_L, cost_L)

d = where(valid, d_sub, filled_int)

disp_u8 = clip(d).astype(uint8)

guide_q = ((guide_bgr >> 4) << 4)

return weightedMedianFilter(
  joint=guide_q,
  src=disp_u8,
  r=15)
```

Basic results

Image	BPR	Time
Tsukuba	3.41%	0.177 s
Venus	0.31%	0.196 s
Teddy	10.09%	0.336 s

- Venus: clean plane structure $\rightarrow$ lowest BPR
- Tsukuba: moderate texture, stable result
- Teddy: occlusion + thin structure $\rightarrow$ harder
- Cones / hidden results: checked using TA score sheet

Why CodaLab runtime varies

- Same code, different grader load
- Thread scheduling affects wall time
- Cost volume is memory-bandwidth heavy
- Cache effects vary by machine
- BPR stable; runtime less stable

```
_MAX_WORKERS = min(os.cpu_count() or 4, 8)

parallel loops:
  1. hamming_cost_volume
  2. aggregate / WGIF

memory use:
  cost volume = (D+1, H, W) float32
```

Bonus: pipeline overview

- Dataset: TartanAir `japanesealley/Hard/P000`
- Reuse `computeDisp` output
- Fuse many frames instead of showing one noisy depth map

Bonus: dataset layout

- Synthetic: stereo + GT depth + GT pose all in one
- Intrinsics constant across V1
  - $f_x = f_y = 320$, $c_x = 320$, $c_y = 240$
  - baseline 0.25 m, image 640x480
- Sky / inf marked ≈ 65504 (float16 max)
- Pose: NED body axes (will fix next slide)

```
data/.../japanesealley/Hard/P000/
  image_left/  000XXX_left.png
  image_right/ 000XXX_right.png
  depth_left/  000XXX_left_depth.npy
  pose_left.txt   # 7 cols per row:
                  # tx ty tz qx qy qz qw
```

Bonus: pose NED $\rightarrow$ OpenCV ($\star$ key fix)

- TartanAir: **NED body** axes
  - x forward, y right, z down
- Open3D / OpenCV: **camera** axes
  - x right, y down, z forward
- Skip $\rightarrow$ mesh floats 3–15 m above path
- Fix: right-multiply by fixed permutation

$$T_{wc}^{\mathrm{CV}} = T_{wc}^{\mathrm{NED}} \cdot M_{\mathrm{NED}\rightarrow\mathrm{CV}}$$

Sanity: $\|t_L - t_R\| = 0.2500\,\mathrm{m}$

$$M_{\mathrm{NED}\rightarrow\mathrm{CV}} = \begin{pmatrix} 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

```
load_poses(traj_dir, indices):
  rows = loadtxt(pose_left.txt)
  for i in indices:
    R = quat_to_R(qx, qy, qz, qw)
    T = build_4x4(R, t)
    out.append(T @ M_NED2CV)
  return out
```

Bonus: disparity → depth + invalid mask

- Triangulation: $Z = f_x \cdot B/d$
- **Two invalid bands** must both be zeroed
  - sky: `depth >= 65000`
  - far: `depth > depth_trunc`
- Open3D treats `depth=0` as invalid (skip)
- Else: clips far points → phantom wall at 15 m

```
disparity_to_depth(disp, fx, B):
  mask = disp >= 1.0
  depth[mask] = fx * B / disp[mask]
  return depth  # float32 metres

prepare_depth(depth, depth_trunc):
  out = depth.copy()
  out[out >= SKY_THRESHOLD] = 0
  out[out > depth_trunc]    = 0
  return out
```

Bonus: TSDF parameter rationale

- `ScalableTSDFVolume`: sparse voxel hash, scales to outdoor scenes
- `voxel_size = 0.05 m`
  - alley width ∼10–20 m
  - 5 cm keeps walls sharp
- `sdf_trunc = 4 × voxel`
- `depth_trunc = 15 m`
  - stereo noise $\propto Z^2/(f_x B)$

```
TSDFConfig:
  voxel_size  = 0.05   # metres
  sdf_trunc   = 0.20   # 4x voxel
  depth_trunc = 15.0   # metres
  depth_scale = 1000   # m -> uint16 mm

make_volume(cfg):
  return ScalableTSDFVolume(
    voxel_length=cfg.voxel_size,
    sdf_trunc=cfg.sdf_trunc,
    color_type=RGB8,
  )
```

Bonus: per-frame integration

- Pack RGB + depth into `RGBDImage`
- depth (m) $\rightarrow$ `uint16` millimetres
- Open3D wants extrinsic $= T_{cam \leftarrow world}$
  - invert our $T_{world \leftarrow cam}$
- TSDF weights samples by viewing angle
- Per-frame stereo noise cancels across views

```
integrate_frame(vol, Il, depth,
                K, T_world_cam, cfg):
  rgb = BGR2RGB(Il)
  depth_clip = clip(depth, 0, trunc)
  depth_mm   = (depth_clip * 1000).uint16
  rgbd = RGBDImage(rgb, depth_mm)

  K_o3d = PinholeCameraIntrinsic(
            W, H, fx, fy, cx, cy)
  extr  = inv(T_world_cam)
  vol.integrate(rgbd, K_o3d, extr)
```

Bonus: mesh extract + first-person render

- Sparse marching cubes from TSDF
- **Follow-trajectory**, not orbit
  - virtual cam at each capture pose
  - video overlaps original frames
  - sidesteps world-up ambiguity
- `OffscreenRenderer`: EGL headless
- Render 2× upscale, K scaled to match

```
mesh = vol.extract_triangle_mesh()
mesh.compute_vertex_normals()

render_follow_trajectory(mesh, poses,
                         K, (W, H)):
  renderer = OffscreenRenderer(rW, rH)
  scene.add_geometry(mesh, defaultLit)
  for T_world_cam in poses:
    T_cam_world = inv(T_world_cam)
    renderer.setup_camera(K_up, T_cam_world)
    frames.append(renderer.render_to_image())
  write_mp4(frames, fps=15, codec=libx264)
```

Bonus: BPR vs TartanAir GT depth

Matcher	BPR @ 1 px	BPR @ 3 px	time / frame
v6 (classical)	20.21%	10.75%	0.36 s
v8 (classical + sub-pixel)	20.26%	10.75%	0.34 s
FoundationStereo	**3.65%**	**1.23%**	0.28 s

- Setup: `japanesealley/Hard/P000` frames 0–99, `max_disp=64`
- Classical hits the texture-poor ceiling at ∼20%
- v8 sub-pixel gains lost at `uint8` API boundary
- Deep model (zero-shot): median BPR 0.91% @ 3 px

Bonus: demo videos

Version	File
v6 (classical baseline)	`out/v6_demo.mp4`
FoundationStereo (deep)	`out/foundation_stereo_demo.mp4`
GT depth (upper bound)	`out/tartanair_p000_gt_100.mp4`

- Same 100-frame TartanAir trajectory
- Same TSDF pipeline; only the matcher changes
- Visual: cleaner walls and ground plane on the deep version